

Natural Computing

LMU Munich
summer term 2025

Thomas Gabor



Algorithm 6 (basic evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process. The process $\mathcal{E}$ continues via an evolutionary algorithm if E has the form

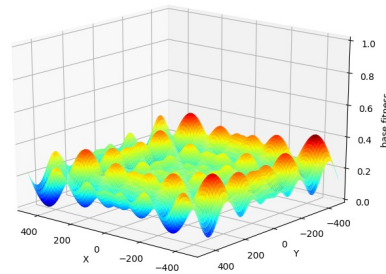
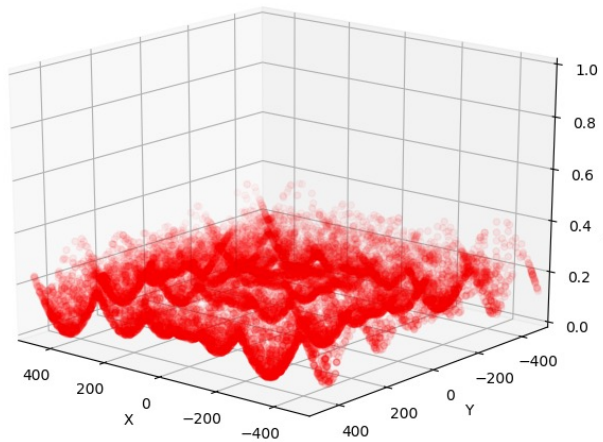
$$E(\langle X_u \rangle_{0 \leq u \leq t}, \tau) = X_{t+1} = \text{selection}(X_t \uplus \text{variation}(X_t))$$

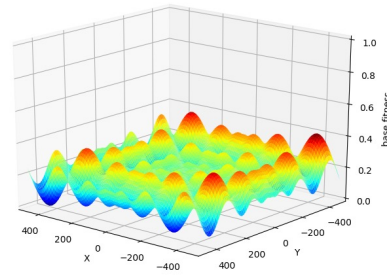
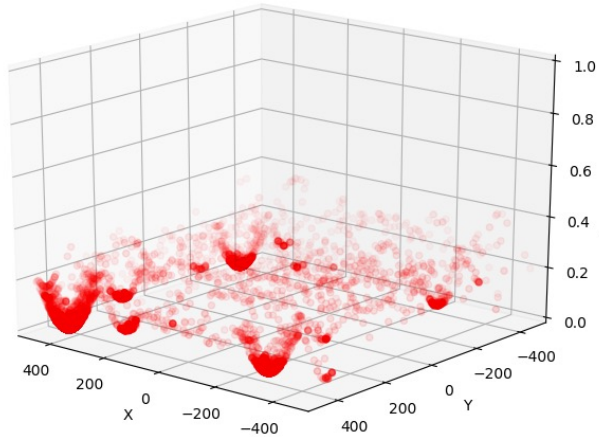
where *selection* and *variation* are possibly randomized or non-deterministic functions so that for any $X \in \wp^*(\mathcal{X})$ it holds that $|\text{selection}(X)| \leq |X|$ and $|\text{selection}(X \uplus \text{variation}(X))| = |X|$.

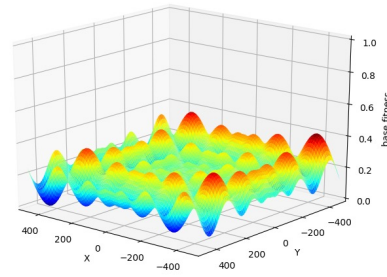
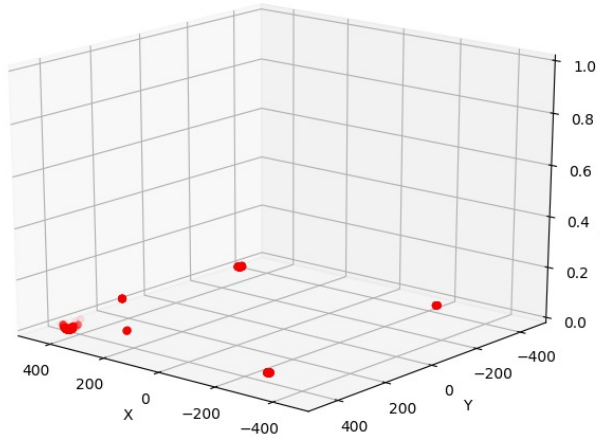
Definition 11 (population-based optimization). Let $\mathcal{X}$ be a state space. Let $\mathcal{T}$ be a target space with total order $\leq$. Let $\tau : \mathcal{X} \rightarrow \mathcal{T}$ be a target function. A tuple $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ is a population-based optimization process iff $X_t \in \wp^*(\mathcal{X})$ for all t and $E : \langle \wp^*(\mathcal{X}) \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \wp^*(\mathcal{X})$ is a possibly randomized, non-deterministic, or further parametrized function so that the population-based optimization run is produced by calling E repeatedly, i.e., $X_{t+1} = E(\langle X_u \rangle_{0 \leq u \leq t}, \tau)$ where X_0 is given externally or chosen randomly.

Definition 11 (population-based optimization). Let $\mathcal{X}$ be a state space. Let $\mathcal{T}$ be a target space with total order $\leq$. Let $\tau : \mathcal{X} \rightarrow \mathcal{T}$ be a target function. A tuple $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ is a population-based optimization process iff $X_t \in \wp^*(\mathcal{X})$ for all t and $E : \langle \wp^*(\mathcal{X}) \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \wp^*(\mathcal{X})$ is a possibly randomized, non-deterministic, or further parametrized function so that the population-based optimization run is produced by calling E repeatedly, i.e., $X_{t+1} = E(\langle X_u \rangle_{0 \leq u \leq t}, \tau)$ where X_0 is given externally or chosen randomly.

Definition 12 (population-based optimization (alternate)). An optimization process $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ is called population-based iff $\mathcal{X}$ has the form $\mathcal{X} = \wp^*(\mathcal{Y})$ for some other state space $\mathcal{Y}$.







Variation



image sources:

en.wikipedia.org/wiki/Mutation#/media/File:Darwin_Hybrid_Tulip_Mutation_2014-05-01.jpg

www.heise.de/ct/artikel/Die-Woche-Microsoft-und-Linux-1283059.html

Natural Computing, summer term 2025, LMU Munich

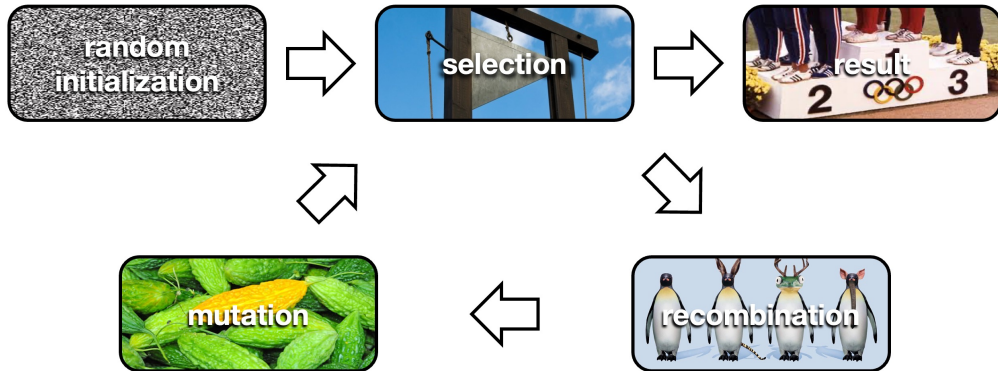


image sources:

www.bostonmagazine.com/news/2015/07/30/boston-2024-winners-losers

en.wikipedia.org/wiki/Mutation#/media/File:Darwin_Hybrid_Tulip_Mutation_2014-05-01.jpg

www.heise.de/ct/artikel/Die-Woche-Microsoft-und-Linux-1283059.html

<https://phys.org/news/2019-10-guillotine-cruel-poisoning.html>

Recombination

Mutation

Migration/Hypermutation

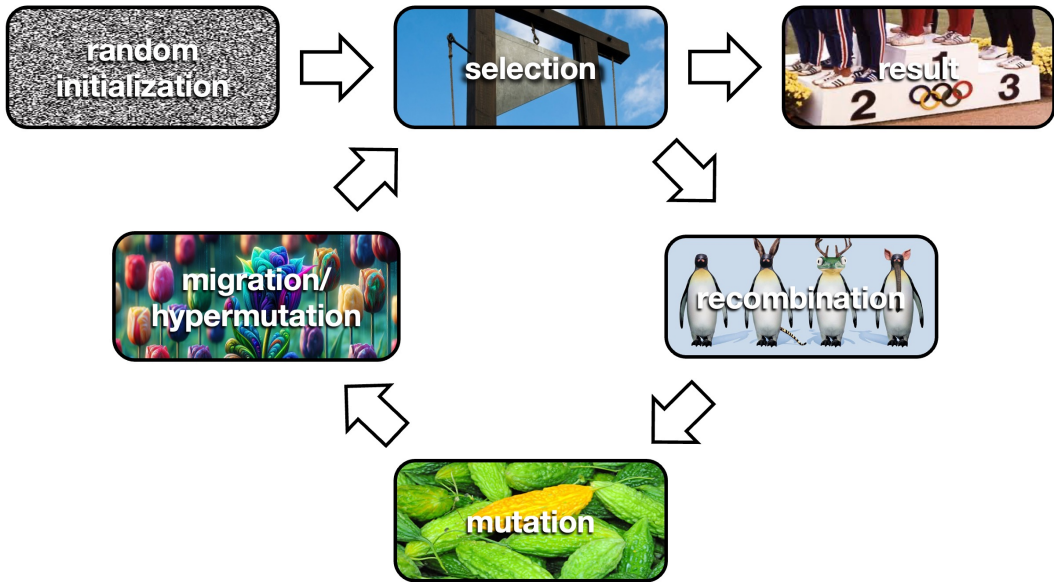


image sources:

www.bostonmagazine.com/news/2015/07/30/boston-2024-winners-losers
en.wikipedia.org/wiki/Mutation#/media/File:Darwin_Hybrid_Tulip_Mutation_2014-05-01.jpg
www.heise.de/ct/artikel/Die-Woche-Microsoft-und-Linux-1283059.html
<https://phys.org/news/2019-10-guillotine-cruel-poisoning.html>
chat.openai.com

Natural vs. Artificial Selection

Target Function vs. Fitness Function

Points of Selection

Types of Selection

cut-off selection

Types of Selection

Tournament Selection

Types of Selection

Roulette Wheel Selection

Algorithm 7 (typical evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process.

- Let $\sigma_N^{survivors}, \sigma_N^{parents} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be (possibly randomized or non-deterministic) selection functions that return $N \in \mathbb{N}$ individuals, i.e., $|\sigma_N(X)| = N$ and $\sigma_N(X) \subseteq X$ for all X . They may possibly depend on $\mathcal{E}$ (most commonly ϕ). Let $\varrho_N^{mutants} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be a special selection function that, given a population X , returns a random subset $X' \subseteq X$ so that $|X'| = N$. Note that $\varrho_{|X|}(X) = X$ for all X .
- Let *mutate* : $\mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) single-individual mutation function. We write *mutate* $[\cdot] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the per-individual application of *mutate*, i.e., *mutate* $[X] = \{\text{mutate}(x) : x \in X\}$.
- Let *recombine* : $\mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) two-parent recombination function. We write *recombine* $[\cdot] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the random-pairing application of *recombine*, i.e., *recombine* $[X] = \{\text{recombine}(x_1, x_2)\} \uplus \text{recombine}[X \setminus \{x_1, x_2\}]$ with $x_1, x_2 \sim X$ and *recombine* $[\{x\}] = \emptyset$ for all x as well as *recombine* $[\emptyset] = \emptyset$.
- Let *migrate* : $\emptyset \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) individual creation function. We write *migrate* $_N : \emptyset \rightarrow \wp^*(\mathcal{X})$ with $N \in \mathbb{N}$ for the iterated application of *migrate*, i.e., *migrate* $_N[\cdot] = \{\text{migrate}()\} \uplus \text{migrate}_{N-1}[\cdot]$ with *migrate* $_0[\cdot] = \emptyset$.

The process $\mathcal{E}$ continues via a typical evolutionary algorithm if E has the form

$$\begin{aligned} E(\langle X_u \rangle_{0 \leq u \leq t}, \phi) = X_{t+1} = & \sigma_{|X_t|}^{survivors}(X_t \\ & \uplus \text{mutate}[\varrho_M^{mutants}(X_t)] \\ & \uplus \text{recombine}[\sigma_{2C}^{parents}(X_t)] \\ & \uplus \text{migrate}_H[\cdot]) \end{aligned}$$

where $M \in \mathbb{N}$ is the number of mutants produced per generation, $C \in \mathbb{N}$ is the number of children produced per generation, and $H \in \mathbb{N}$ is the number of migrants (sometimes called hypermutants) generated per generation. Especially M is often expressed as a rate for random choice within the population, i.e., $M = \lceil m \cdot |X_t| \rceil$ where $m \in \mathbb{R}$ is called mutation rate.

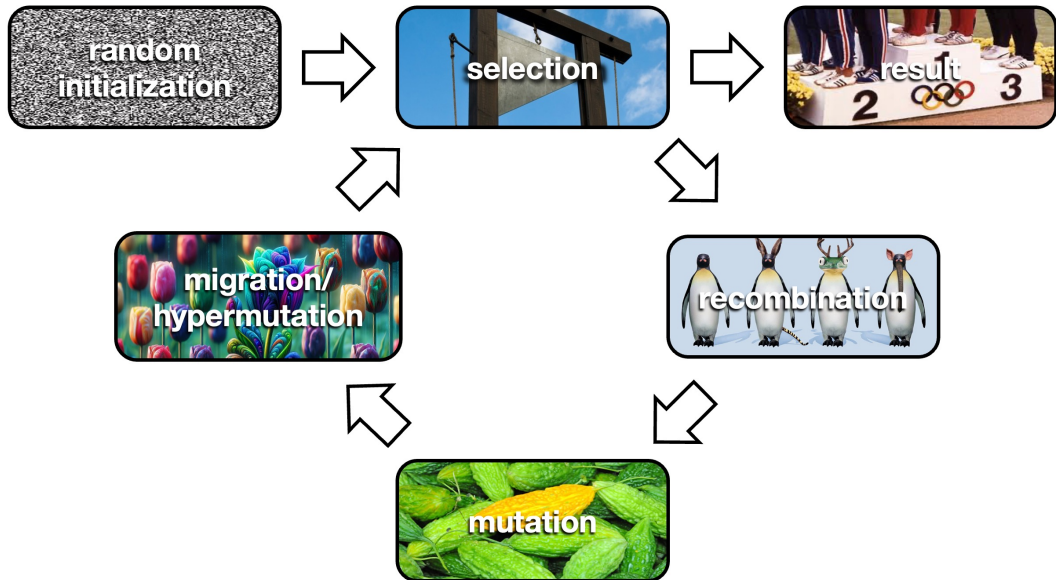


image sources:

www.bostonmagazine.com/news/2015/07/30/boston-2024-winners-losers

en.wikipedia.org/wiki/Mutation#/media/File:Darwin_Hybrid_Tulip_Mutation_2014-05-01.jpg

www.heise.de/ct/artikel/Die-Woche-Microsoft-und-Linux-1283059.html

<https://phys.org/news/2019-10-guillotine-cruel-poisoning.html>

chat.openai.com

Exploration/Exploitation for EAs

Diversity

Algorithm 8 (diversity-aware evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be the process of an evolutionary algorithm (cf. Alg. 7). $\mathcal{E}$ is called a diversity-aware evolutionary algorithm if the used fitness function $\phi : \mathcal{X} \rightarrow \mathcal{T}$ has the form

$$\phi(x) = (1 - \zeta) \cdot \tau(x) + \zeta \cdot \delta(x)$$

where $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is the underlying target function of the given optimization problem, $\delta : \mathcal{X} \rightarrow \mathcal{T}$ is a (possibly randomized, non-deterministic, or further parametrized) function that estimates the diversity that a given individual contributes to the population, and $\zeta \in [0; 1] \subset \mathbb{R}$ is called a diversity weight.

Note that according to Thm. 4 and Gabor et al. [2] the diversity estimate

$$\delta(x) = \frac{1}{|X_t|} \cdot \sum_{x' \in X_t} |x' - x|$$

is generally suitable for $\mathcal{X} = \mathbb{R}^n$ or $\mathcal{X} = \mathbb{Z}^n$ for some n .

Algorithm 8 (diversity-aware evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be the process of an evolutionary algorithm (cf. Alg. 7). $\mathcal{E}$ is called a diversity-aware evolutionary algorithm if the used fitness function $\phi : \mathcal{X} \rightarrow \mathcal{T}$ has the form

$$\phi(x) = (1 - \zeta) \cdot \tau(x) + \zeta \cdot \delta(x)$$

where $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is the underlying target function of the given optimization problem, $\delta : \mathcal{X} \rightarrow \mathcal{T}$ is a (possibly randomized, non-deterministic, or further parametrized) function that estimates the diversity that a given individual contributes to the population, and $\zeta \in [0; 1] \subset \mathbb{R}$ is called a diversity weight. Note that according to Thm. 4 and Gabor et al. [2] the diversity estimate

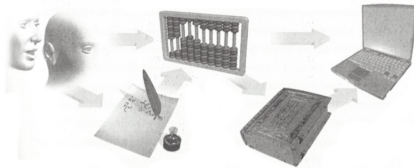
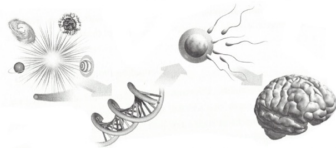
$$\delta(x) = \frac{1}{|X_t|} \cdot \sum_{x' \in X_t} |x' - x|$$

is generally suitable for $\mathcal{X} = \mathbb{R}^n$ or $\mathcal{X} = \mathbb{Z}^n$ for some n .

Theorem 4 (Wineberg-Oppacher diversity [4]). All measures of the diversity of an individual x in a population X are related to the pairwise distance between individuals in X as measured by a suitable norm (Euclidean for real-valued representations, Hamming for symbolic representations, e.g.).

The Ideal Fitness Function?

(Social)
Behavior



“The history of the universe can be thought of as a sequence of information processing revolutions, each of which builds on the technology of the previous ones.”

Seth Lloyd. *Programming the Universe*. Vintage Books, 2006.

Organisms

Learning During a Lifetime



image sources:

www.bostonmagazine.com/news/2015/07/30/boston-2024-winners-losers

en.wikipedia.org/wiki/The_Thinker#/media/File:Le_Penseur_in_the_Jardin_du_Musée_Rodin,_Paris_14_June_2015.jpg

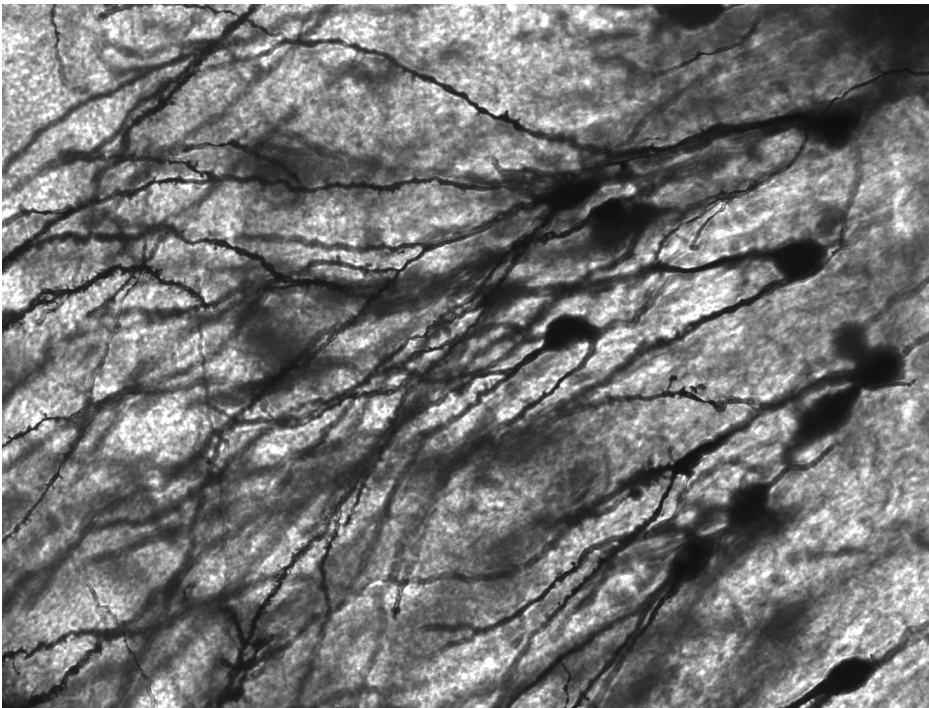
www.boredpanda.com/jumping-cats/

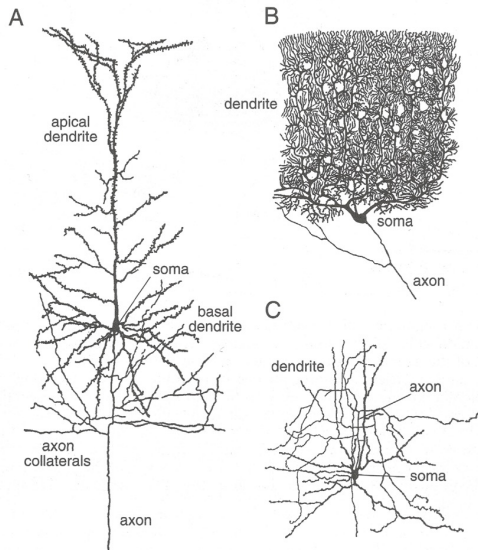
kinder.wdr.de/tv/wissen-macht-ah/bibliothek/kuriosah/bibliothek-daumen-hoch-100.html

www.medicalnewstoday.com/articles/320289.php#carry_message

Natural Computing, summer term 2025, LMU Munich

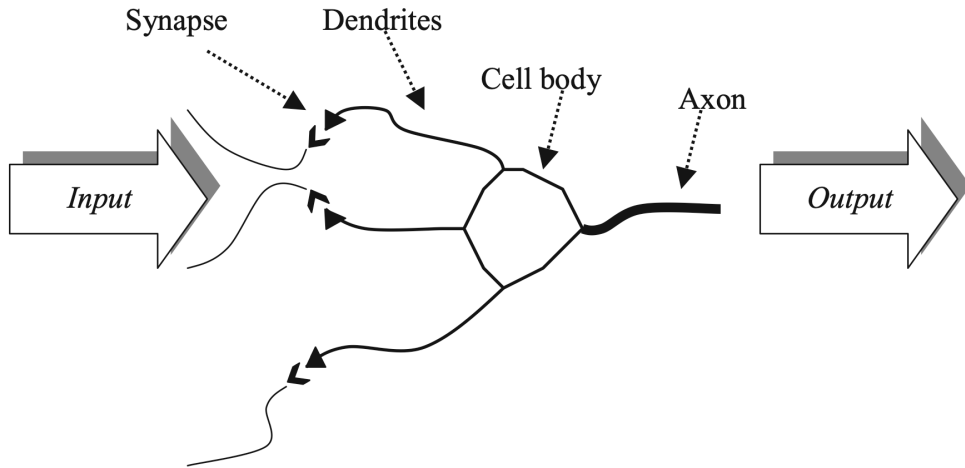
source: commons.wikimedia.org/wiki/File:Gyrus_Dentatus_40x.jpg



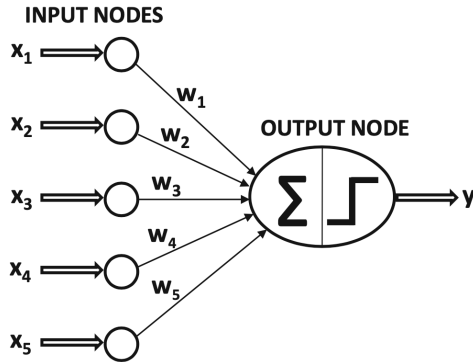


source: Dayan, Abbott.
 Theoretical Neuroscience:
 Computational and Mathematical
 Modeling of Neural Systems.
 MIT Press, 2001.

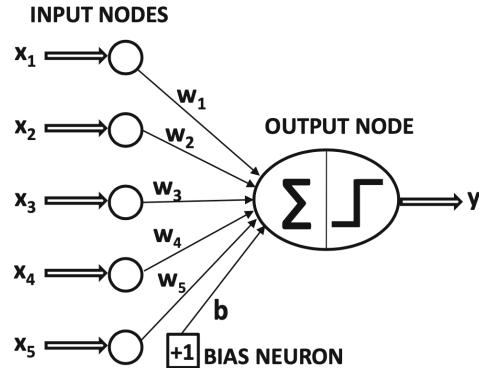
Figure 1.1 Diagrams of three neurons. (A) A cortical pyramidal cell. These are the primary excitatory neurons of the cerebral cortex. Pyramidal cell axons branch locally, sending axon collaterals to synapse with nearby neurons, and also project more distally to conduct signals to other parts of the brain and nervous system. (B) A Purkinje cell of the cerebellum. Purkinje cell axons transmit the output of the cerebellar cortex. (C) A stellate cell of the cerebral cortex. Stellate cells are one of a large class of interneurons that provide inhibitory input to the neurons of the cerebral cortex. These figures are magnified about 150-fold. (Drawings from Cajal, 1911; figure from Dowling, 1992.)



Artificial Neurons

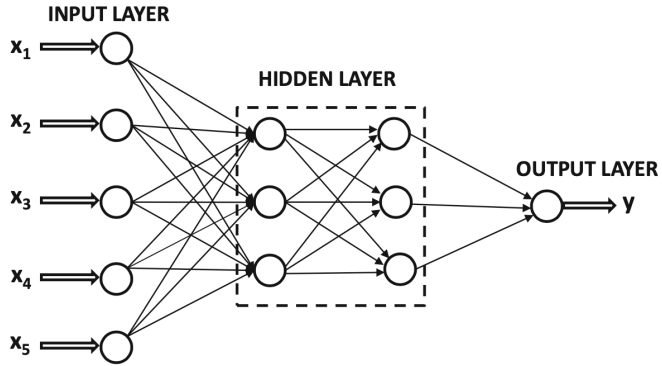


(a) Perceptron without bias

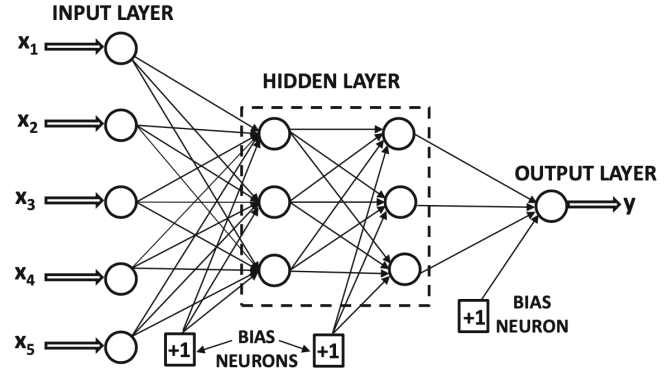


(b) Perceptron with bias

Neural Networks



(a) No bias neurons



(b) With bias neurons

More Organisms



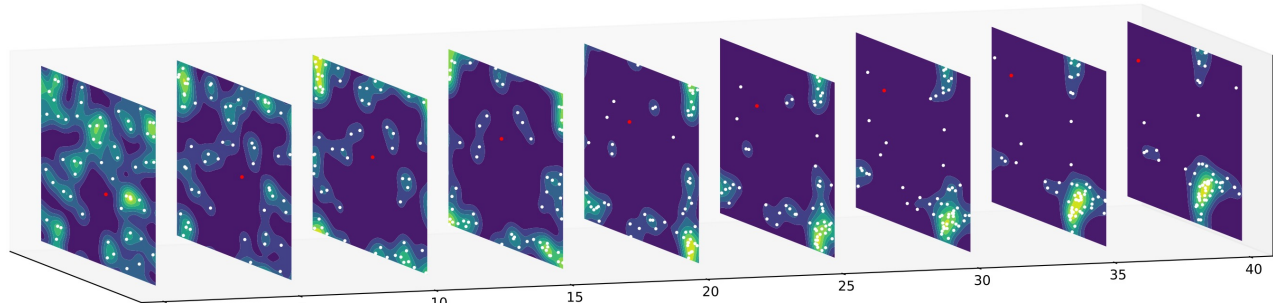
source: en.wikipedia.org/wiki/Swarm_behaviour#/media/File:Auklet_flock_Shumagins_1986.jpg



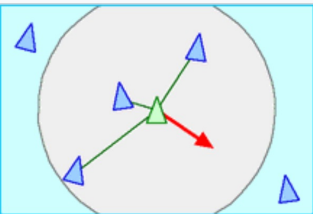
source: en.wikipedia.org/wiki/File:Moofushi_Kandu_fish.jpg

Hahn et al. Emergent Escape-based Flocking Behavior
using Multi-Agent Reinforcement Learning. 2019.

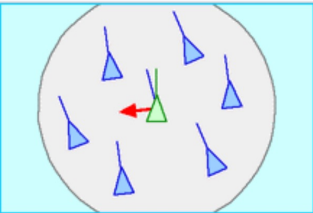




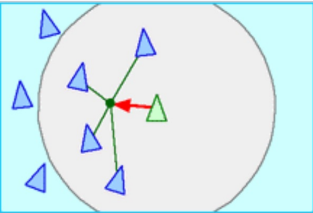
Rules applied in simple Boids



Separation



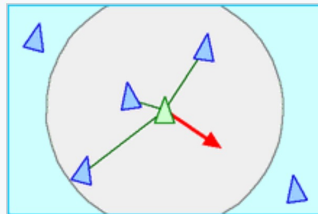
Alignment



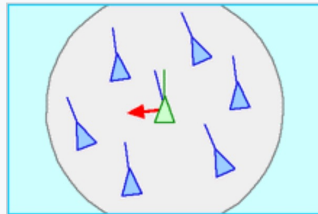
Cohesion

Guess what we
can do with
it?

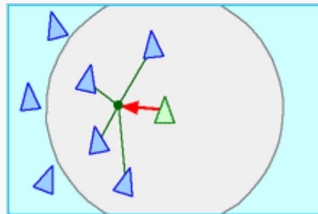
Rules applied in simple Boids



Separation



Alignment



Cohesion

source: en.wikipedia.org/wiki/Boids

Particle Swarm Optimization (PSO)

Algorithm 8.1: Canonical Particle Swarm Algorithm

```
for each particle i in the population do
    Initialise its location by randomly selecting values;
    Initialise its velocity vector to small random values close to zero;
    Calculate its fitness value;
    Set initial  $p_i^{\text{best}}$  to the particle's current location;
end
Determine the location of  $g^{\text{best}}$ ;
repeat
    for each particle i in turn do
        Calculate its velocity using (8.1);
        Update its position using (8.2);
        Measure fitness of new location;
        if fitness of new location is greater than that of  $p_i^{\text{best}}$  then
            | Revise the location of  $p_i^{\text{best}}$ ;
        end
    end
    Determine the location of the particle with the highest fitness;
    if fitness of this location is greater than that of  $g^{\text{best}}$  then
        | Revise the location of  $g^{\text{best}}$ ;
    end
until terminating condition;
```

$$v_{id}(t+1) = v_{id}(t) + c_1 r_{1d}(p_{id}^{\text{best}}(t) - x_{id}(t)) + c_2 r_{2d}(g_d^{\text{best}}(t) - x_{id}(t)) \quad (8.1)$$

$$x_{id}(t+1) = x_{id}(t) + v_{id}(t+1) \quad (8.2)$$

Honeybee Foraging Optimization

Algorithm 10.1: Canonical Honeybee Foraging Optimisation Algorithm

Randomly locate n foraging bees in the search space;
Evaluate the fitness of each of these locations;
Store location of best solution;

repeat

 Select the m best locations (patches) for neighbourhood search ($m \leq n$);
 Recruit foragers to search in the vicinity of these locations (number of foragers assigned to each location is proportional to the fitness of the location) and evaluate the fitness for each forager;
 Select the fittest bee for each patch;
 Assign the remaining bees to search randomly and evaluate their fitness;
 Update location of best solution if necessary;

until *terminating condition*;

Continuous Ant Colony Optimization

Algorithm 9.2: Continuous Ant Foraging Algorithm

Select value of $x_{\min}$ randomly from the range of allowable values for each element $(1, \dots, n)$ of the real-valued solution encoding;

Set σ for each element $(1, \dots, n)$ of the solution encoding;

repeat

for *each ant k in turn* **do**

 Construct a solution $(x_1, \dots, x_n)$ by making a random draw from $\text{PDF}_1, \dots, \text{PDF}_n$;

 Measure quality of solution;

end

 Update the $x_{\min}$ values for each PDF if a new best-so-far solution has been found;

 Update the σ values for each PDF;

until *terminating condition*;

$$\text{Pheromone}(x) = e^{\frac{-(x-x_{\min})^2}{2\sigma^2}} \quad (9.16)$$

Detour: Ant Algorithms for Graph Problems

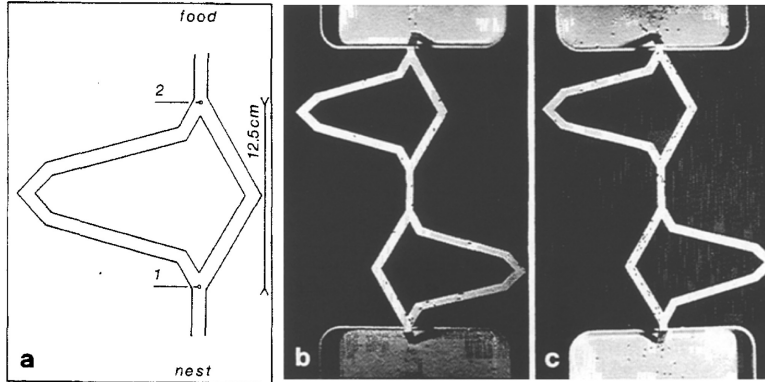


Fig. 1. A colony of *I. humilis* selecting the short branches on both modules of the bridge; a) one module of the bridge, b) and c): photos taken 4 and 8 min after placement of the bridge

source: Goss, Aron, Deneubourg, Pasteels.
Self-Organized Shortcuts in the Argentine
Ant. Naturwissenschaften 76. Springer, 1989.

Detour: Ant Algorithms for Graph Problems

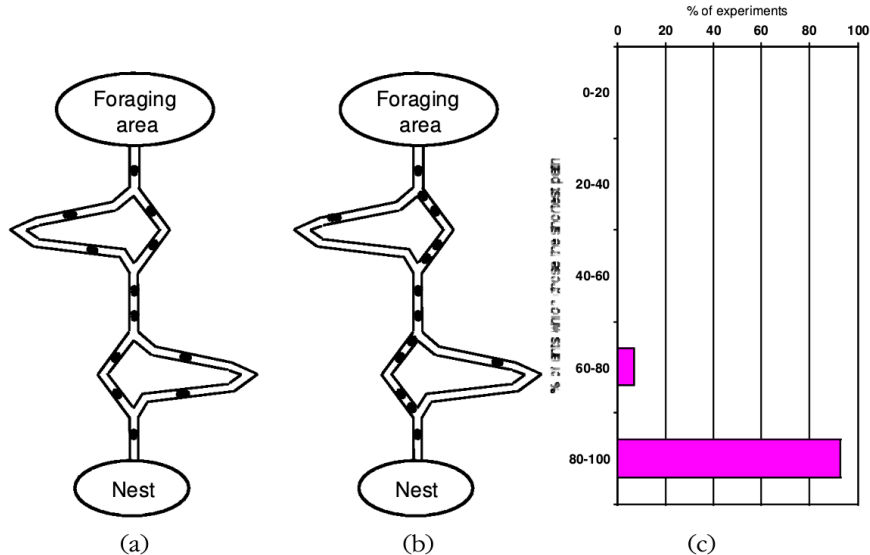


Figure 2. Double bridge experiment. (a) Ants start exploring the double bridge. (b) Eventually most of the ants choose the shortest path. (c) Distribution of the percentage of ants that selected the shorter path. After Goss et al. 1989 [60].

source: Dorigo, Di Caro, Gambardella. Ant Algorithms for Discrete Optimization. Artificial Life. MIT, 1999.

Detour: Ant Algorithms for Graph Problems

Stigmergy

pheromones

Detour: Ant Algorithms for Graph Problems

```
1 procedure ACO_Meta_heuristic()
2   while (termination_criterion_not_satisfied)
3     schedule_activities
4       ants_generation_and_activity();
5       pheromone_evaporation();
6       daemon_actions(); {optional}
7   end schedule_activities
8 end while
9 end procedure

10 procedure ants_generation_and_activity()
11   while (available_resources)
12     schedule_the_creation_of_a_new_ant();
13     new_active_ant();
14   end while
15 end procedure
```

```
16 procedure new_active_ant() {ant lifecycle}
17   initialize_ant();
18    $\mathcal{M}$  = update_ant_memory();
19   while (current_state  $\neq$  target_state)
20      $\mathcal{A}$  = read_local_ant_routing_table();
21      $\mathcal{P}$  = compute_transition_probabilities( $\mathcal{A}$ ,  $\mathcal{M}$ , problem_constraints);
22     next_state = apply_ant_decision_policy( $\mathcal{P}$ , problem_constraints);
23     move_to_next_state(next_state);
24     if (online_step-by-step_pheromone_update)
25       deposit_pheromone_on_the_visited_arc();
26       update_ant_routing_table();
27     end if
28      $\mathcal{M}$  = update_internal_state();
29   end while
30   if (online_delayed_pheromone_update)
31     evaluate_solution();
32     deposit_pheromone_on_all_visited_arcs();
33     update_ant_routing_table();
34   end if
35   die();
36 end procedure
```

source: Dorigo, Di Caro, Gambardella. Ant Algorithms for Discrete Optimization. Artificial Life. MIT, 1999.

2	Simulated Annealing	SA	1983
3	Stochastic Diffusion Search	SDS	1989
4	Ant Colony Optimization	ACO	1992
5	Particle Swarm Optimization	PSO	1995
6	Photosynthetic Algorithm	PA	1999
7	Clonal Selection Algorithm	CSA	2000
8	Sheep Flocks Heredity Model Algorithm	SFHM	2001
9	Marriage in Honey Bees	MHB	2001
10	Harmony Search	HS	2001
11	Bacterial Foraging Optimization	BFO	2002
12	Fish-swarm Algorithm	FSA	2002
13	Bacterial Chemotaxis	BC	2002
14	Hysteretic Optimization	HO	2002
15	Electromagnetism-like Optimization	EM	2003
16	BeeHive		2004
17	Bee Colony Optimization	BCO	2005
18	Space Gravitational Algorithm	SGA	2005
19	Particle Collision Algorithm	PCA	2005
20	The Bees Algorithm		2006
21	Glowworm Swarm-based Optimization	GSO	2006
22	Honey-bees Mating Optimization	HBMO	2006
23	Shuffle frog-leaping Algorithm	SFLA	2006
24	Cat Swarm Optimization	CSO	2006
25	Group Search Optimizer	GSO	2006
26	Saplings Growing up Algorithm	SGA	2006
27	Invasive Weed Optimization	IWO	2006
28	Big Bang - Big Crunch	BB-BC	2006
29	Artificial Bee Colony	ABC	2007
30	Bacterial Swarming Algorithm	BSA	2007
31	Monkey Search	MS	2007
32	Intelligent Water Drop	IWD	2007
33	River Formation Dynamics	RFD	2007
34	Central Force Optimization	CFO	2007

261	Bald Eagle Search	BES	2019
262	Raccoon Optimization Algorithm	ROA	2019
263	Sea Lion Optimization Algorithm	SLeO	2019
264	Electric Fish Optimization	EFO	2019
265	Urban Pigeon-Inspired Optimiser		2019
266	Xerus Optimization Algorithm	XOA	2019
267	Artificial Ecosystem-based Optimization	AEO	2019
268	Physarum-energy Optimization Algorithm	PEO	2019
269	Artificial Coronary Circulation System	ACCS	2019
270	Artificial Ecosystem-based Optimization	AEO	2019
271	Nuclear Reaction Optimization	NRO	2019
272	Whirlpool Algorithm	WA	2019
273	Electrostatic Discharge Algorithm	ESDA	2019
274	Targeted Showering Optimization	TSO	2019
275	Atom Search Optimization	ASO	2019
276	Henry Gas Solubility Optimization	HGSO	2019
277	Artificial Electric Field Algorithm	AEFA	2019
278	Evolutionary Centers Algorithm	ECA	2019
279	Ohm's Law Optimization algorithm	OLO	2019
280	Electron Radar Search Algorithm	ERSA	2019
281	Spherical Search Optimizer	SSO	2019
282	Manta Ray Foraging Optimization	MRFO	2020
283	Tunicate Swarm Algorithm	TSA	2020
284	Black Widow Optimization	BWO	2020
285	Black Widow Optimization Algorithm	BWO	2020
286	Collective Animal Behavior	CAB	2020
287	Side-Blotched Lizard Algorithm	SBLA	2020
288	Marine Predators Algorithm	MPA	2020
289	Shuffled Shepherd Optimization Algorithm	SSOA	2020
290	Water Strider Algorithm	WSA	2020
291	Fertile Field Algorithm	FFA	2020
292	Equilibrium Optimizer	EO	2020
293	Newton Metaheuristic Algorithm	NMA	2020